Assignment 4 – Persistent Multi-Page Posting System

CMPT 353 – Full Stack Web Development

Name: Olisa Emeka Kenneth
Student ID: 11402253
Date:12th Febreuary 2026

---

1. Project Overview

This project implements a full-stack persistent posting system using:

- React (Frontend)
- Express (Backend)
- MySQL (Database)
- Docker Compose (Containerization)
- Nginx (Frontend production server)

The application allows users to:

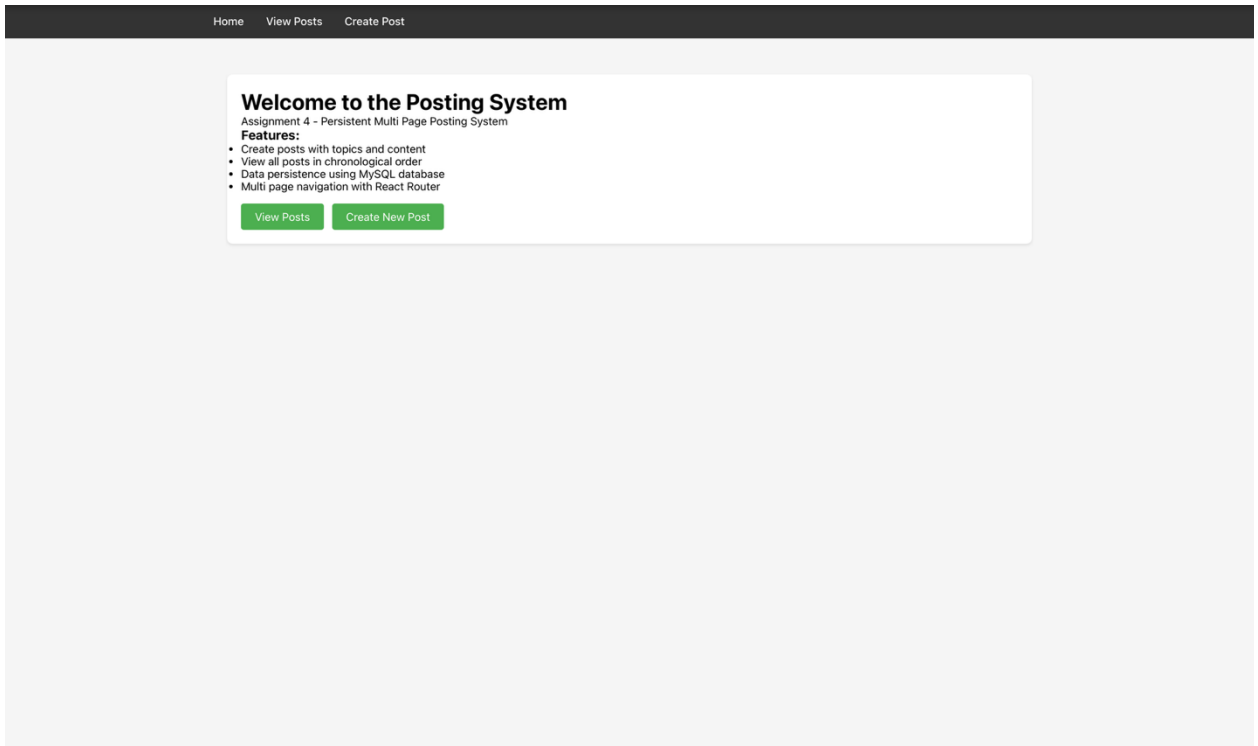
- Create posts with a topic and content
- View all posts in reverse chronological order
- Store data persistently in a MySQL database
- Navigate between multiple pages using React Router

All services are containerized and run using Docker Compose.

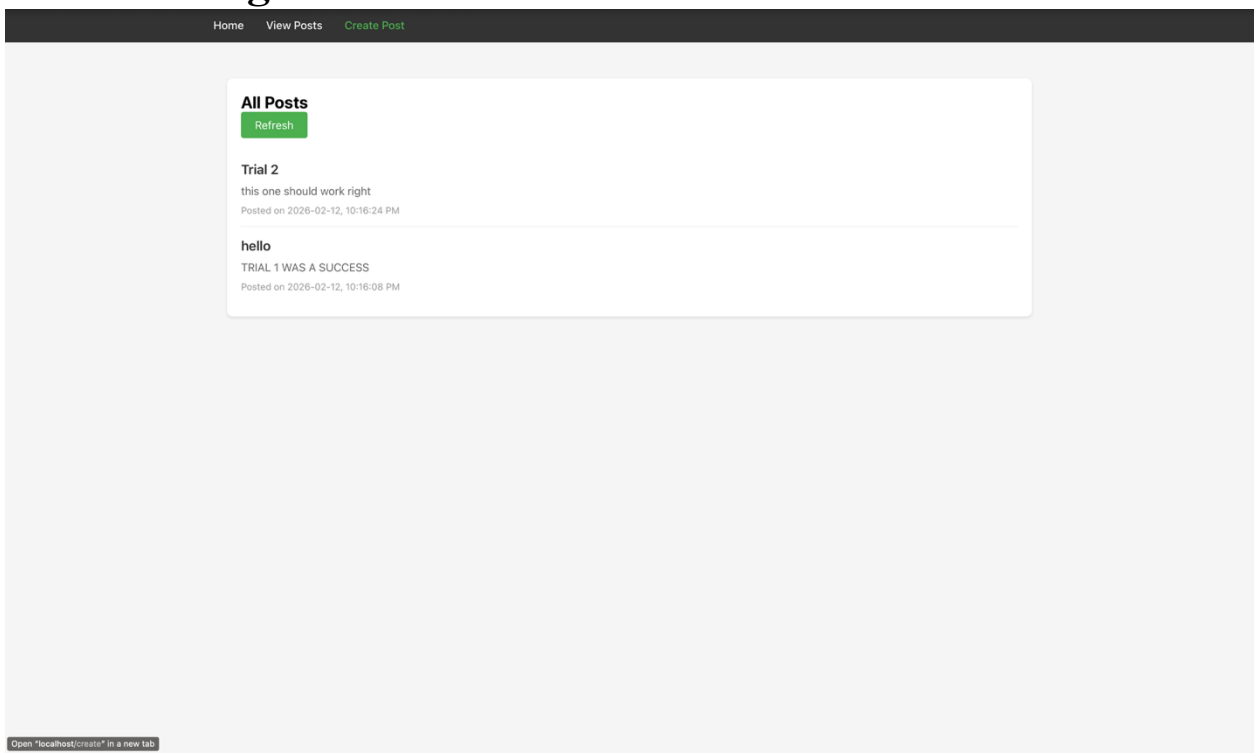
---

2. Screenshots

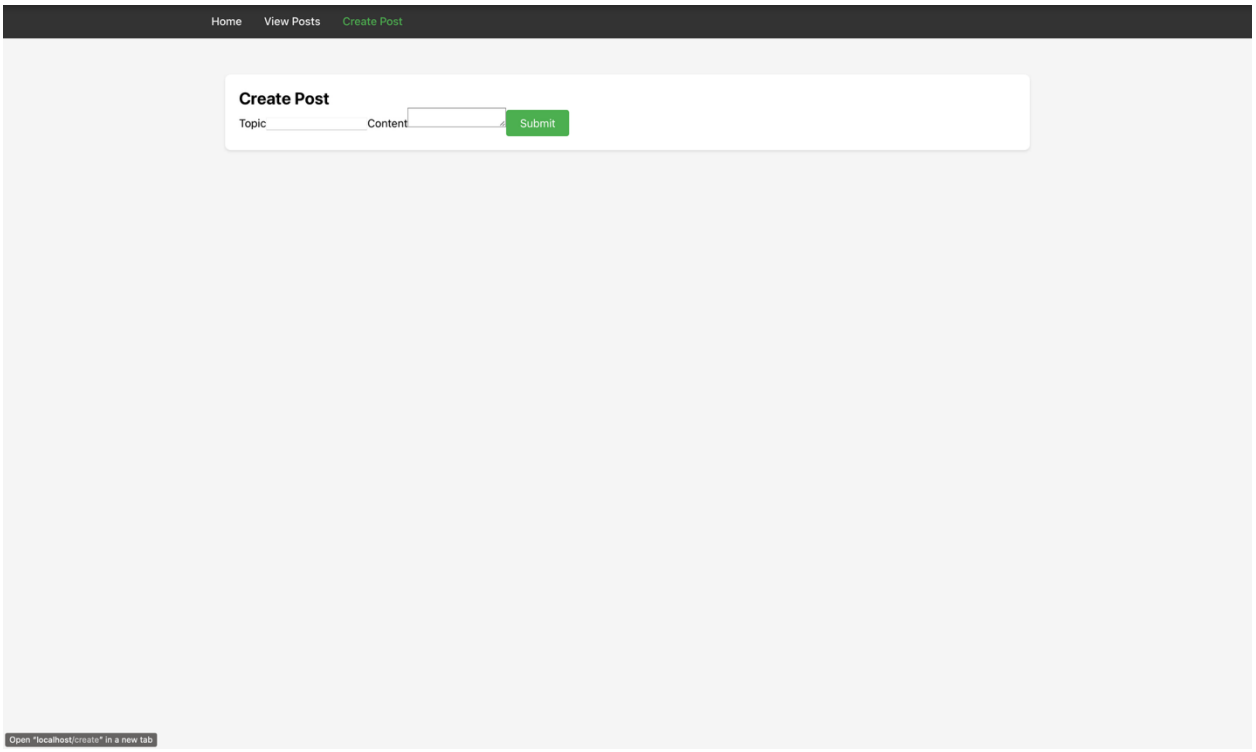
2.1 Landing Page



2.2 Posts Page



2.3 Create Post Page



---

3. Overall Testing Approach

Testing was performed at both the backend and frontend levels.

3.1 Backend Testing

Backend testing focused on verifying:

- Correct API endpoint functionality
- Proper JSON formatting
- Input validation
- Error handling
- Database persistence

The following endpoints were tested:

GET /api/posts

Purpose:

- Retrieve all posts sorted by newest first.

Tested by:

- Visiting http://localhost/api/posts
- Using loadtest for stress testing

Expected behavior:

- Returns JSON array of posts
- Sorted in reverse chronological order

Result:

- Correct JSON returned
- Proper ordering confirmed

POST /api/posts

Purpose:

- Create a new post.

Request body format:

```
{
  "topic": "Example",
  "data": "Content"
}
```

Expected behavior:

- HTTP 201 response
- Post inserted into MySQL database

Result:

- Post successfully stored and returned

Validation Testing

Cases tested:

- Missing topic
- Missing data

Expected behavior:

- HTTP 400 error returned

Result:

- Proper validation error triggered

3.2 Frontend Testing

Frontend testing verified:

- Navigation between pages using React Router
- Fetch requests correctly retrieving data
- Form submission flow
- Display of newly created posts
- Proper handling of API errors

Tests performed:

- Navigation from Home → Posts → Create
- Creating a post and verifying it appears immediately
- Refreshing page to confirm persistence
- Stopping backend container to observe frontend behavior

Result:

- All flows functioned correctly
- API errors were visible in browser console when backend was down

---

4. Functional Test Cases

Test Case	Input	Expected Result	Actual Result
Valid Post Creation	topic: "Test", data: "Hello"	Post saved and displayed	Works correctly
Missing Topic	topic: "", data: "Hello"	HTTP 400 error	Validation triggered

Test Case	Input	Expected Result	Actual Result
Missing Data	topic: "Test", data: ""	HTTP 400 error	Validation triggered
Very Long Data	5000+ characters	Post stored successfully	Stored correctly
Backend Down	Stop backend container	Frontend shows fetch error	Error observed

All functional test cases behaved as expected.

---

5. Load Testing

Load testing was performed using the loadtest tool.

5.1 GET Load Test

Command used:

```
npx loadtest -n 1000 -c 10 http://localhost/api/posts
```

Results

- Total Requests: 1000
- Concurrent Clients: 10
- Total Errors: 0
- Total Time: 0.433 seconds
- Mean Latency: 20.4 ms
- Effective Requests per Second: 2309

Latency distribution:

- 50% of requests completed under 15 ms
- 90% completed under 22 ms
- 95% completed under 111 ms
- 99% completed under 126 ms
- Longest request: 133 ms

5.2 Analysis

The backend handled concurrent traffic without errors.
Latency remained low under moderate load.
No failed requests were observed, indicating stable API performance.

---

6. Persistence Testing

Persistence was tested as follows:

1. Created multiple posts.
2. Stopped containers using:

```
docker compose down
```

3. Restarted containers using:

```
docker compose up
```

4. Verified posts were still available.

Result:

- Posts persisted successfully due to MySQL Docker volume.

---

7. Challenges and Solutions

Challenge 1: Nginx 404 for /api routes

Solution:

- Added reverse proxy configuration to forward `/api` to backend container.

Challenge 2: CSS not appearing

Solution:

- Ensured index.css was imported in main.jsx.
- Rebuilt Docker with `--no-cache`.

Challenge 3: Docker build caching

Solution:

- Used `docker compose build --no-cache` to force rebuild.

Challenge 4: MySQL initialization timing

Solution:

- Used `depends_on` in docker-compose.
- Ensured proper database initialization script.

---

8. Conclusion

The application successfully meets all of my assignment requirements:

- Multi-page React frontend
- Express REST API backend
- MySQL database persistence
- Docker Compose containerization
- Production frontend build
- Functional and load testing

All required screenshots and testing documentation are included.